Project 3 – Hungry Lizard Crossing: Analysis

COP4634 Systems & Networks I – Fall 2026 – Lance Penaflor (LP) and Jay McIntosh (JM)

1. Problem and Solution

Many lizard threads (and two cat threads) share one resource, the driveway. If more than `MAX_LIZARD_CROSSING` (4) lizards are on it at once, a cat sees them and the program ends. The shared counters `numCrossingSago2MonkeyGrass` and `numCrossingMonkeyGrass2Sago` are read by cats and written by lizards, so they are a race condition unless protected. Lizards also must not busy-wait, and the maximum allowed number (4) should be able to cross together.

The solution uses one counting **semaphore** (`driveway`) initialised to 4 permits. A lizard takes a permit before crossing and returns it after arriving, so at most 4 lizards are ever on the driveway, and all 4 permits can be in use at once. A blocked lizard sleeps inside the semaphore (no spinning). A **mutex** (`counterLock`) protects every read and write of the crossing counters, including the cat's check. The main thread sleeps `WORLDEND` seconds, sets `running = 0`, then joins every lizard and cat thread before destroying the synchronization objects.

2. Changes Made (all marked `// LP JM` in the source)

- **`Semaphore` class** – C++11 has no semaphore, so a counting semaphore was built from a `mutex` + `condition_variable`. `wait()` blocks until a permit is available; `post()` returns one and notifies a waiter.
- **Globals** – `counterLock` (mutex), `driveway` (semaphore), and `maxCrossing` (largest total seen crossing, for the report).
- **`sago2MonkeyGrassIsSafe` / `monkeyGrass2SagoIsSafe`** – call `driveway->wait()`.
- **`madeIt2MonkeyGrass` / `madeIt2Sago`** – call `driveway->post()`.
- **`crossSago2MonkeyGrass` / `crossMonkeyGrass2Sago`** – increment and decrement the counters under `counterLock`, update `maxCrossing`, and keep the lock released during the `sleep(CROSS_SECONDS)`.
- **`Cat::catThread`** – the check of both counters is done under `counterLock`.
- **`Lizard::lizardThread`** – implements the life cycle: sleep, sago→grass safe, cross, made it, eat, grass→sago safe, cross, made it.
- **`Cat::run` / `Cat::wait`, `Lizard::~Lizard`** – create, join and free the threads.
- **`main`** – creates the semaphore, the lizard and cat objects, starts the threads, sleeps `WORLDEND`, sets `running = 0`, joins all threads, prints the maximum crossing count, and deletes everything. `WORLDEND` is 180 in the submitted file.
- **Extra credit (`lizardsUni.cpp`, `UNIDIRECTIONAL 1`)** – adds direction control on top of the above: two "lightswitch" counters (one per direction, each with its own mutex). The first lizard to enter a direction waits on the `roomEmpty` semaphore; the last one to leave posts it. A `turnstile` semaphore makes new arrivals queue behind a waiting opposite-direction lizard so neither direction starves. The existing "Crash!" checks never fire.

3. Results

Compiled with `g++ -g -Wall -std=c++11 … -pthread`; no errors or warnings. Each run exited normally (the cats never saw too many lizards).

Program	WORLDEND (s)	Lizards	Cats	Max lizards crossing	Lizards safe?
lizards (bidirectional)	30	5	2	4	Yes
lizards (bidirectional)	30	20	2	4	Yes
lizards (bidirectional)	30	50	4	4	Yes
lizards (bidirectional)	60	20	2	4	Yes
lizards (bidirectional)	180	20	2	4	Yes
lizardsUni (unidirectional)	30	20	2	4	Yes
lizardsUni (unidirectional)	30	50	4	4	Yes
lizardsUni (unidirectional)	60	20	2	4	Yes

The maximum is exactly 4 in every run: the limit is never exceeded, and 4 lizards do cross together.

4. Issues Encountered

- **No semaphore in C++11.** `std::counting_semaphore` needs C++20 and the Makefile uses `-std=c++11`, so the semaphore was written from a mutex and condition variable (the wait uses a predicate, avoiding lost wakeups and spurious-wakeup bugs).
- **Cat exiting while holding the lock.** The cat's `exit(-1)` path happens while `counterLock` is held; that is fine because the process ends, but every normal path unlocks.
- **Holding the lock while sleeping.** Sleeping the crossing time under `counterLock` would serialize all crossings; the lock is released before `sleep(CROSS_SECONDS)` so several lizards cross at once.
- **Shutdown.** Lizards can be mid-sleep or mid-meal when `running` becomes 0, so the program takes a few extra seconds to join (a 180 s run took about 196 s). Every permit is always returned, so no thread stays blocked.
- **Unidirectional starvation.** With only the per-direction counters, a steady stream of same-direction lizards could keep the driveway from ever emptying and starve the other direction, so the turnstile was added to stop new arrivals from joining a group once an opposite-direction lizard is waiting.
- **File naming.** The starter file was named `lizard.cpp`; it was renamed to `lizards.cpp` to match the assignment and the Makefile.